

Section 1: About You

Goals and Objectives

Why do you want to do a GSoC project with Oppia?	I began contributing to Oppia-Android in August 2024, and it has been an incredibly fulfilling journey. Through this experience, I have not only strengthened my Android development skills but also become part of a welcoming and supportive community. What truly resonates with me is Oppia's mission to make quality educational content with engaging elements like cartoons, narratives to make learning more interactive and effective and accessible to everyone. Contributing to a GSoC project with Oppia would be a great opportunity to make a meaningful impact while continuing to grow as a developer.	
What do you hope to learn/achieve during GSoC?	During GSoC, I plan to deepen my understanding of advanced Android development concepts, explore scalable architectures for large platforms, and improve my ability to write clean, maintainable code that adheres to industry best practices. Additionally, I hope to enhance my communication skills through collaboration with mentors and the open-source community.	
Which Oppia teams have you collaborated with, and what have you done on those teams?	Android CLaM	Here are some <i>non-coding</i> contributions that I would like to highlight: • Reviewed PRs. [Link]. • Identified and Reported an Issue in the App. [Link] [Link]. • Guided new contributors. [Link]. • Documented Clear Steps to Reproduce an Existing Issue. [Link]. • Facilitated the Reopening of a Closed Issue. [Link].
Contact information	Email	subhajitmallick879@gmail.com
	GitHub	https://github.com/subhajitxyz
	LinkedIn	https://www.linkedin.com/in/subhajit-mallick-63a5a0243/
Preferred method of communication	Email Google Chat (part of Android CLaM Team)	
Social media handles (optional) <i>This field is optional. We ask for it so that we can tag you in GSoC-related posts if your application is accepted!</i>	Linkedin	
Which timezone will you primarily be in during the summer?	Indian Standard Time (+5:30 GMT).	

(If you are a student) When are your school holidays?	My summer holidays will start from June 5th to August 15th this year. Consequently, I will have a significant amount of free time from June until almost August.
What other obligations might you need to work around during the summer?	During the summer, I have no other obligations that I need to work around.
Planned time commitment	My planned time commitment will be a. 5-6 hours/day when on holiday. b. 4 hours/day when in college without exams. c. 2 hours/day when in college with exams. From June to August, I anticipate dedicating nearly 300 hours in total. Additionally, I am open to dedicating time on weekends if needed.

Project Timeframe

Note: *Oppia will only be offering a single GSoC coding period timeframe this year, starting on **Jun 2**. All work for Milestone 1 must be completed and submitted by **Jul 4** for internal feedback (with any revisions due by **Jul 18**), and all work for Milestone 2 must be completed and submitted by **Aug 27** for internal feedback (with any revisions due by **Sep 10**). We will not be able to extend these deadlines.*

Coding period	I will adhere to the above deadlines.
----------------------	---------------------------------------

Communication Channels

Note: *The Oppia team places a high emphasis on communication, and we have found that daily contact between contributors and mentors is important for helping keep projects on track. This is why we ask that contributors send short daily updates to their mentors explaining what they have done, where they are stuck, and what they plan to do next.*

I can commit to sending daily updates to my mentor by email, each day I work during the GSoC period.	Yes.
In addition to the above: how often, and through which channel(s), do you plan on communicating with your mentor?	My plan is to share daily updates with my mentor to maintain clear communication and ensure a smooth project workflow. For routine discussions and minor queries, I prefer using Google Chat or email. Additionally, I propose holding two meetings per week with my mentor for flexibility, using Google Meet or any other preferred platform.

Contributor Briefing

Note: A couple of days after the selection results are announced, there will be a mandatory contributor briefing for accepted contributors. Please confirm whether you will be able to attend the briefing.

If accepted, I will be able to attend the contributor briefing at 3 pm UTC on May 10.	Yes.
--	------

Section 2: About Your Project

Project Details

Project title <i>(should match the one on the Project Ideas list)</i>	4.1. Flashbacks
Project size <i>(should match the options on the Project Ideas list)</i>	Medium (~175 hours)
Why did you choose this project?	Firstly, it aligns perfectly with my interests and aspirations. Tackling a project of this complexity will not only enhance my coding skills but also improve my workflow efficiency. Over the past few months, as I've explored this project in depth, I've gained a strong understanding of its codebase, which further motivates me to contribute meaningfully. Secondly, I have previously worked on a similar issue and proposed a solution in PR. This experience has given me valuable insights into the project, and I believe I can leverage that knowledge to develop an even better solution, especially for a large and complex codebase.

Required Skills (Project-Specific)

GENERAL	
I can figure out the root cause of an issue, and communicate it effectively	Yes. • Getting some unexpected result on implementing #171

using a debugging doc . ¹	
I can debug and fix CI failures. ²	In PR # 5626 , I fixed CI failures related to static checks, lint issues, and script validations by identifying root causes and applying the necessary corrections.
I can make changes to GitHub Actions workflows. ³	N/A.
ANDROID	
Build the app and install it on a local device or emulator. Then verify that you can (a) build a non-test library target locally, (b) run a unit test locally, and (c) play through the app locally. ⁴	(a) link. (b) link. (c) link.
Write new Kotlin code with unit tests. ⁵	• Fix part of #5485: Create means for verifying Fragment Arguments • Fix #5431 : Todo Checks Should Check Exclusively Against Issues • Fix #5612 Talk back is not reading Arabic language in Preferred Audio Language
Change Android UIs, write tests for them, and manually verify that they work. ⁶	• Fixes #5572 : Softer redirection for learners who need to revisit earlier cards • Fixes #5600: Add developer options menu support for creating multiple profiles • Fix #5381 Talkback reads 14 underscore in a question in What is a ratio chapter

¹ To develop this skill: Tackle an issue where the root cause is unclear, identify the root cause in a comment on the issue thread (perhaps with a debugging doc explaining how you figured it out) and submit a PR that fixes it. Fixing CI failures also counts. You might find [this tutorial](#) helpful.

² To develop this skill: See issues labelled "[CI breakage](#)" on GitHub, or look at [CI failures in develop](#) (or  signs [here](#)) and figure out how to fix them. [This tutorial](#) (for Web) might also be useful.

³ To develop this skill: Please see the list of issues for the [dev workflow team](#). However, note that this skill is not needed for most projects and there are limited issues available for it. If this affects your project, [please contact the org admins](#).

⁴ To develop this skill: Please follow the [Bazel setup instructions](#) to get your local setup working. If you run into issues, ask a question in [Oppia Android's discussions board](#).

⁵ To develop this skill: essentially all [CLaM issues](#) will require Kotlin and testing work. Focus on issues that aren't marked as "backlog".

⁶ To develop this skill: Any CLaM issue that involves fixing something that can be observed from within the app itself will apply to this skill. See [this list](#) as a high-level reference (but note that some of those only affect tests). If you're unsure whether a particular issue applies, ask in the Oppia Android GSoC Q&A Discussion board.

Section 3: Proposal Details

Problem Statement

Target Audience	Learners – who will be able to view the correct answer or revisit their previously submitted answers related to the same concept when they get stuck on a question, and directly return to that question without reattempting intermediate ones. Lesson creators and educators – who design lessons and aim to improve the learning experience by reinforcing key concepts effectively.
Core User Need	As a learner, when I encounter a difficult question and get stuck, I need a way to view the correct answer along with my previous question-and-answer submissions related to the same concept. This will help me understand how I solved it previously without having to redo all the intermediate questions I've already completed. By doing this, I can reinforce my understanding of the concept I struggled with while continuing to progress smoothly through the lesson.

Section 3.1: WHAT

Key User Stories and Tasks that will be implemented

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that"</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
1	Learner Engagement	As a stuck learner who can acknowledge that I'm stuck, I need a happy path to get "unstuck" when dealing with a difficult question so that I can learn the concept correctly.	The learner answers the question incorrectly and does not get hints.	Mocks
			Learner acknowledges that they are "stuck" when attempting to answer a question.	Mocks
			Learner sees a Flashback Card with a detailed explanation of the previous question.	Mocks
			Learner returns to the question at which acknowledgment of being stuck occurred.	Mocks PRD
			Learner reattempts.	Mocks PRD

Testing Plan

Critical user journeys

User Type	User Story Description	Test definition		
		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
Learner	1. As a stuck learner who can acknowledge that I'm stuck on a subsequent question , I want to decline to view the flashback card when offered	- Install the app. - Log into profile. - Click on a topic card from the list. - Navigate through the lessons to reach a difficult question. - Submit a wrong answer. - The "Revisit Previous Question" button is displayed.	Click the "Revisit Previous Question" button.	A pop-up dialog box will be displayed (suggesting, "Need help?" with two buttons: 1. Not Now 2. Continue).
			Click the "Not Now" button.	Learner is taken back to the question where they got stuck so they can attempt it again.
	2. As a stuck learner who can acknowledge that I'm stuck on a subsequent question , I want to view the flashback card when offered.	Click the "Revisit Previous Question" button.	Click the "Continue" button on pop-up dialog box.	The following are displayed: - A flashback Screen (with toolbar heading "Review Previous Question") - the previous question, - Oppia's suggested correct answer, - the user's submitted correct answer, - a "Return to Question" button.
			Click on "Return to Question."	Learner is taken back to the question where they got stuck so they can attempt it again.
	3. As a stuck	Click the	Click the "Close" icon	Learner is taken back to

	learner who can acknowledge that I'm stuck on a subsequent question. Then, see the flashback card when offered, and then want to attempt the question I was stuck on.	"Revisit Previous Question" button. - Click the "Continue" button on the pop-up dialog box. - The Flashback Screen is displayed.	on the left top corner.	the question where they got stuck so they can attempt it again.
--	---	--	--------------------------------	--

Technical Requirements

Additions/Changes to the Web Server Interface

No additions/changes to the Web Server interface are required.

New/Changed Calls by the Web Frontend /Android Client to the Web Server Interface

No new/changed calls by the web Frontend /Android client to the Web Server Interface are required.

Web Server Storage Model Additions/Changes

No additions/changes to Web Server Endpoint Contracts are required.

UI Screens/Components

#	ID	Description of new UI component	i18n required?	Mock/spec links	What A11y support will be implemented?
1.	"Revisit Previous Question" Button	Provide learners to get a way to unstuck on questions where they struggle.	Yes. All user facing strings need to be translated.	Mock	When Talkback is enabled, a user will be able to read and respond using their pre-configured accessibility settings
3.	Flashback Screen	Provide: - A reddish-orange toolbar titled "Review Previous Question" with an audio icon and a 3-dot menu icon. - A correct solution to the question where the learner is stuck.	Yes. All user facing strings need to be translated.	Mock	When Talkback is enabled, a user will be able to read and respond using their pre-configured accessibility settings

		- The previous question and the correct answer submitted by the user, linked to their misconception of the current question they are stuck on. - An option to return to the current question after reviewing the answer. - An audio option that allows the learner to select their preferred language for listening.			
4.	"Need Help?" Pop-up Dialog	This dialog allows the learner to: - View the flashback card (containing the previous question and the learner's submitted answer) by clicking the "Continue" button. - Return to the current question without reviewing the answer by clicking the "Not Now" button.	Yes. All user facing strings need to be translated.	Mock	When Talkback is enabled, a user will be able to read and respond using their pre-configured accessibility settings

Data Handling and Privacy

The project does not retain any sensitive user data. Users need not worry about their privacy in this regard because the data we will collect will not contain any user PII.

Connection to Existing Work

Changes in Learner Conversational Flow:

The introduction of a new button("Revisit Previous Question") for opening the Flashback card, and displaying feedback follows patterns already established in the codebase.

Similar implementations for showing other buttons("Continue", "Submit") and other views exist in [StatePlayerRecyclerViewAssembler](#), and [StateFragmentPresenter](#), which handle dynamic UI updates and learner interactions.

Flashback Card Implementation:

We will display the flashback card using the [state_recycler_view](#) in state_fragment.xml.

Additionally, a new toolbar will be introduced in [exploration_activity.xml](#), featuring a distinct title and color to visually differentiate the flashback mode.

We will introduce the necessary views, ViewModels, and support functions required to render the UI of the flashback card. The [compute\(\)](#) method of StatePlayerRecyclerViewAssembler will be used to generate the list of ViewModels based on the appropriate ephemeral state for the flashback card.

Enhancing the Look of Submitted Answers:

Currently, for multiple-choice and item-selection answers, we display only the options selected by the user in the submitted answer section.

To improve clarity and user experience, we will enhance the design and presentation by displaying **all available choices** for each question, while **clearly highlighting the user's selected option(s)**. This will help users better review their answers and understand them in the context of all possible options, aligned with the design shown in the [mock](#).

To support displaying multiple choices submitted answers along with the correct answer in a structured manner, the implementation will leverage existing layouts and patterns. The required design and logic are already present in [selection_interaction_item.xml](#), [multiple_choice_interaction_items.xml](#), and [item_selection_interaction_items.xml](#), along with their corresponding ViewModels, ensuring seamless integration into the learner's response flow.

Other Requirements

N/A

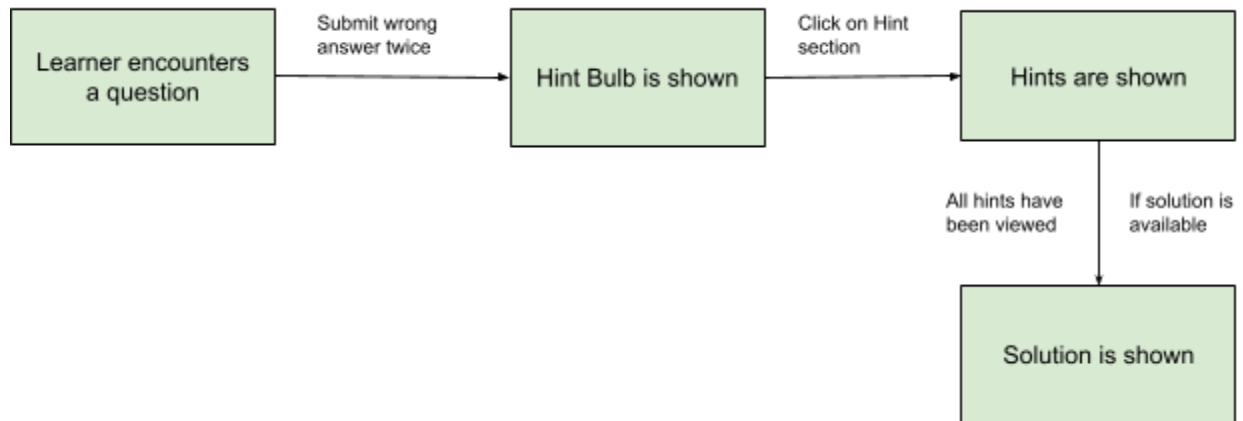
Section 3.2: HOW

Existing Status Quo

The Oppia Android app supports hints and solutions to guide learners when they submit incorrect answers. If a learner answers incorrectly twice or stays inactive for a set duration, a hint bulb appears. Hints are displayed sequentially, and if a solution is available, it is shown only after all hints have been viewed. Some questions include multiple hints, while others have none.

[HintHandler](#) manages hint visibility, while [ExplorationProgressController](#) determines when hints should be displayed. The [HintHandlerProdImpl](#) ensures consistency with the Oppia web platform, handling delays and hint availability based on learner interaction.

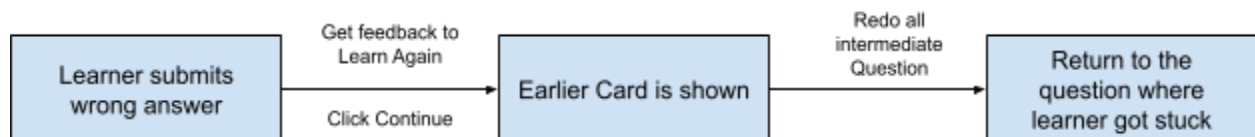
The following flowchart helps us to understand the current user flow.



Additionally, the app allows learners to revisit an earlier card if they struggle with a previously learned concept. However, the current implementation forces them to re-answer all intermediate questions before returning to their original point, making the experience frustrating and repetitive.

[AnswerClassificationController](#) classifies user responses based on Oppia's interaction rule engine, determining the appropriate outcome for each answer. Based on AnswerOutcome, the [ExplorationProgressController](#) determines which state to pass to [StateDeck](#), which then manages the learner's navigation by showing the appropriate next state.

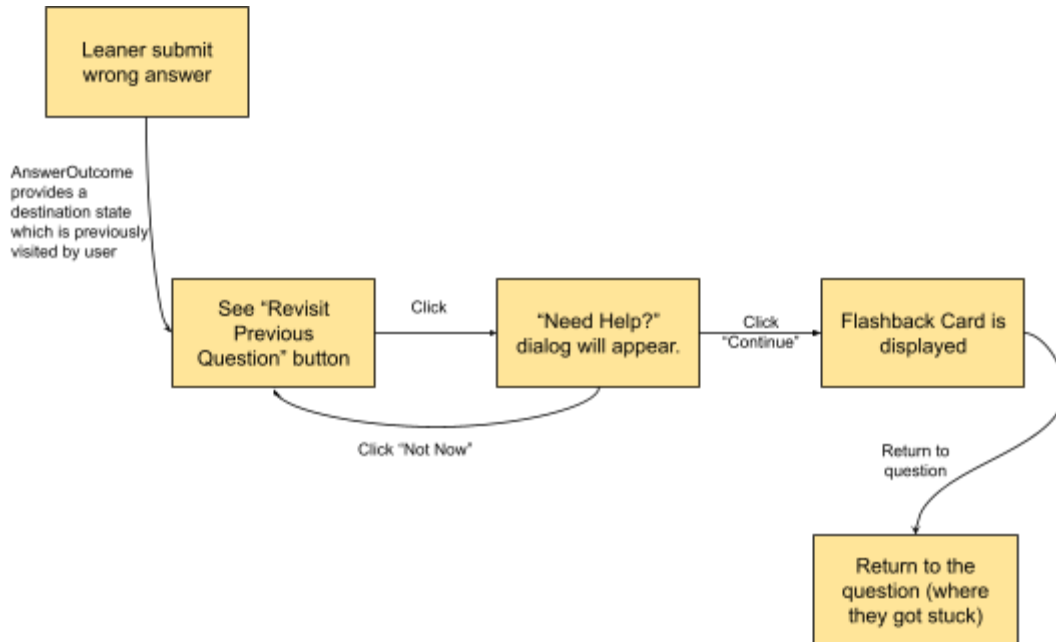
The following flowchart helps us to understand the current user flow.



The app renders submitted answers for multiple-choice and item-selection interactions using HTML generation.

Solution Overview

This solution introduces a Flashback feature that allows learners to revisit previously solved question answers (linked to their misconceptions) when they get stuck on a question. By reviewing an earlier answer tied to their misconception, learners can reinforce their understanding before retrying the question. After viewing the flashback, they can return directly to the current question without redoing intermediate steps, ensuring a seamless learning experience.



Additionally, the solution improves the visual presentation of submitted answers for multiple-choice and item selection interactions by replacing HTML generation with a natively rendered approach, enhancing UI consistency and integration within the Flashback experience.

Detection for Enabling Redirection Button to Revisit Flashback Screen

To provide learners the option to access Flashback, we first need to determine when to present the ["Revisit Previous Question"](#) button. This button will allow them to view the previously solved question answer linked to their misconception from this lesson.

The following conditions must be met to enable Flashback:

- The learner must submit an incorrect answer.
- The *AnswerOutcome* for the answer must correspond to a state that's not the current state, and this destination state must have been previously visited by the learner.

This check can be implemented in the [submitAnswerImpl](#) function within the *ExplorationProgressController*. If both conditions are satisfied, we will store the destination state name in *AnswerAndResponse* as *state_name_to_revisit*.

*(We will need to introduce `String state_name_to_revisit` in [AnswerAndResponse](#). The `state_name_to_revisit` in *AnswerAndResponse* will indicate 'Revisit Previous Question' button should be displayed.)*

After viewing the Flashback, when the learner returns to the current question, they will see the 'Revisit Previous Question' button along with the feedback message: "Now that you have reviewed the solution to the previous question, let's try again" in the previous response section.

To implement this feature, the "Revisit Previous Question" button and feedback in the previous response section, we will need to:

- Introduce a String `state_name_to_revisit` field to `AnswerAndResponse`.
`State_name_to_revisit` indicates to display the "Revisit Previous Question" button.
- After revisiting Flashback Card, when the learner returns to the latest pending state, we will update the existing `AnswerAndResponse` instance (with **`state_name_to_revisit`** and **`feedback`**) in **`currentDialogInteractions`**. Together, these two fields indicate to show this button in the previous response section. These updates will be implemented in [StateDeck](#).
- Introduce a new XML file, **`flashback_button_item.xml`**, and **`FlashbackButtonViewModel`**. Additional changes will be made in `StatePlayerRecyclerViewAssembler`, such as implementing `addRedirectionButtonSupport()` and [addPreviousAnswers\(\)](#) functions to show the "Revisit Previous Question" button.

The learner will be able to revisit the Flashback by clicking the "Revisit Previous Question" button.

Flashback Card

When the learner clicks the "Revisit Previous Question" button, a confirmation dialog box will be opened. Upon confirming the dialog, the Learner will be redirected to Flashback Card.

To display the Flashback card, we need to introduce new structures in the existing `exploration.proto`:

1. Add a String `state_name_to_revisit` in [AnswerAndResponse](#).
(*`state_name_to_revisit` contains information about which state the learner should be redirected to. The `state_name_to_revisit` determines whether the redirection button is displayed ("Revisit Previous Question").*)
2. Add a **`bool flashback_state`** as [one_of state type](#) in `EphemeralState` proto message.
(*The `flashback_state` will indicate to the UI to display the Flashback Screen.*)

The `bool flashback_state` in `one_of stateType` of `ephemeralState` will be used to:

- Identify whether the flashback card UI will be rendered in the `state_recycler_view`.
- Control the visibility of the default `exploration_toolbar`, hiding it when in flashback mode.
- Display the newly introduced toolbar with the custom title and color.

To support flashback navigation, we will introduce **`moveToFlashback()`** in `ExplorationProgressController` and handle it with **`moveToFlashbackImpl()`** in `ControllerState`. We will add a `ControllerMessage.MoveToFlashback` to carry the request, calculate the `FlashbackState`, and update `getCurrentState()` accordingly. This implementation will help the learner to revisit the Flashback card by clicking the "Revisit Previous Question".

To facilitate navigation from the Flashback card to the stuck question, we will introduce **moveBackToLatest()** in ExplorationProgressController and handle it with **moveBackToLatestImpl()** in ControllerState.

We will add a ControllerMessage.MoveBackToLatest to carry the request, and update getCurrentState() accordingly.

We will use [compute\(\)](#) of **StatePlayerRecyclerViewAssembler** for generating a list of view models from the appropriate ephemeralState for flashback card, along with other supporting functions. The logic for displaying the UI in the flashback card will mirror the existing implementation used in the [state_recycler_view](#) within state_fragment.xml.

To display redirection buttons and the Flashback card UI, we will reuse existing views and introduce new views, ViewModels, and support functions in StatePlayerRecyclerView.

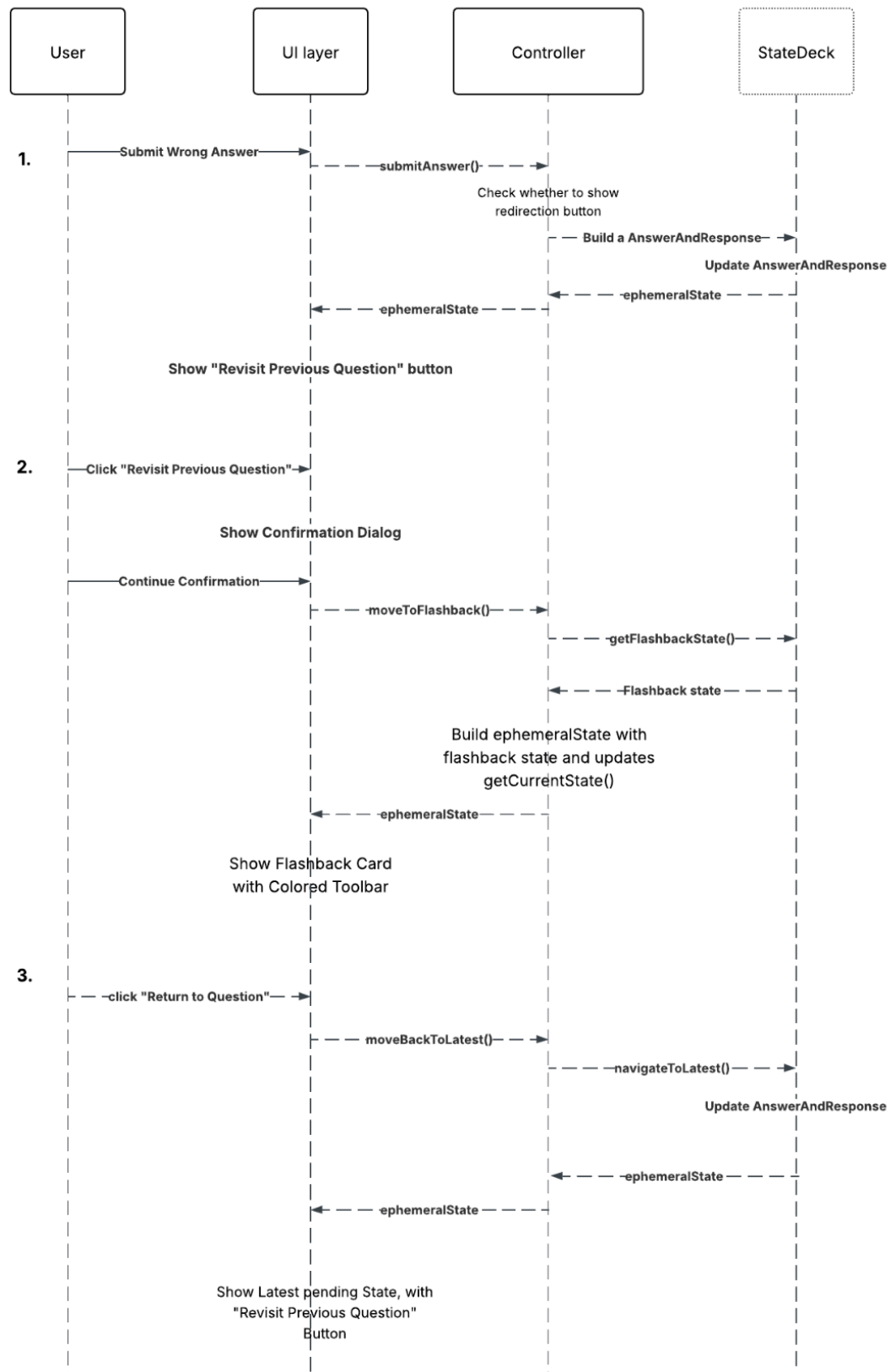
Redirection buttons

- **Revisit Previous Question button** - introduce *Flashback_button_item.xml*, *FlashbackButtonViewModel*, *addRedirectionSupport()*.
- **Return to Question button** - introduce *return_to_question_button_item.xml*, *ReturnToQuestionViewModel*, *addRedirectionSupport()*.

Flashback card UI Components

- **Feedback** - use existing *feedback_item.xml*, *FeedbackViewModel*, *addFeedbackSupport()*.
- **Previous questions** - use existing *content_item.xml*, *ContentViewModel*, *addContentSupport()*,
- **Solution section** - use existing *solution_summary.xml* with some changes and introduce support function *addSolutionSupport()* in *statePlayerRecyclerViewAssembler*.
- **Submitted Answer** - use existing *submitted_answer_item.xml*, *SubmittedAnswerViewModel* and *addPastAnswerSupport()* but need some changes to support enhanced look and ui.
- **Return to question button** - introduce *return_to_question_button_item.xml*, *ReturnToQuestionViewModel*, *addRedirectionSupport()*.
- **New Toolbar**- introduce new toolbar for flashback screen with a different color and title in *exploration_activity.xml*.

Sequence Diagram



Proto Changes During Flashback Lifecycle

Flashback has not been triggered yet

- state_name_to_revisit is not set in AnswerAndResponse.
- explorationProgressController.getCurrentState() returns ephemeralState of state_type PendingState.

After an incorrect answer that has triggered Flashback redirection possibility

- state_name_to_revisit is set in AnswerAndResponse.
- getCurrentState() still returns ephemeralState of state_type PendingState.
- The UI shows a "Revisit Previous Question" button using that field.

Click the redirection button and view the Flashback card

- User clicks the "Revisit Previous Question" button.
- moveToFlashback() is called, which triggers computing FlashbackState.
- explorationProgressController.getCurrentState() returns EphemeralState with state type flashback_state, computed in the previous step.
- StateDeck is unchanged.
- state_name_to_revisit is still present in AnswerAndResponse.

After clicking "Return to question"

- explorationProgressController.getCurrentState() returns EphemeralState with state type pending_state for pending state, which is the state where the learner got stuck.
- No changes to StateDeck.
- state_name_to_revisit is still present in AnswerAndResponse, which will indicate to show "Revisit Previous Question" button in the previous response section.

Enhancing the Look and Feel of the Submitted Answer in FlashbackCard

In the existing app, the submitted answer (for both multiple-choice and item-selection interaction) shows only the user-submitted answer. We can enhance its appearance and usability based on the provided [mock](#).

To improve the visual representation of submitted answers in the Flashback Card, we can use the following approach:

- Add a RecyclerView (question_options_recycler_view) inside [submitted_answer_item.xml](#). This will allow a structured and dynamic display of submitted answers for both multiple-choice and item selection interaction, improving readability.
- Introduce new XML layouts:
 - multiple_choice_submitted_item.xml.
 - Item_selection_submitted_item.xml

These layouts will define how submitted answers appear based on the interaction type.

- Modify [addPreviousAnswers\(\)](#) in `StatePlayerRecyclerViewAssembler`. Pass interaction to `SubmittedAnswerViewModel` to determine the appropriate answer type.
- Enhance Data Handling in [SubmittedAnswerViewModel](#). Compute `List<SelectionSubmittedItemViewModel>` based on `interaction.id`. Mark the correct options for better user understanding. The computed list of answer items will be set in `SubmittedAnswerViewModel`.

`question_options_recycler_view` will fetch the processed list from `SubmittedAnswerViewModel` and display the submitted answer with proper formatting.

A similar type of functionality is also present in the [selection_interaction_item.xml](#) and [SelectionInteractionViewModel](#), which helps in presenting multiple choice options for a question.

Third-Party Libraries

No new third-party libraries are required.

“Service” Dependencies

This feature adds no new service dependencies.

Key High-Level and Architectural Decisions

Decision 1: How to show the previous question and correct answer on Flashback Screen

I have considered the following alternatives:

Alternative 1: Use `state_recycler_view` in `state_fragment.xml` to show new required views for the flashback card.

Alternative 2: Introduce a required view like `question_text`, `feedback_text`, `correct_answer_view` in `state_fragment.xml`.

Among these, I believe that **Alternative 1** is the best approach, because:

- **Scalability:** Introducing separate views for the question, feedback, and correct answer may lead to scalability issues in the future, especially if additional elements need to be incorporated into the `FlashbackCard`. In contrast, using `state_recycler_view` provides a more flexible structure that can accommodate future enhancements with minimal modifications.

- **Complexity:** While implementing `state_recycler_view` with new views, viewmodels may initially seem complex, the existing app already has a similar implementation in `state_fragment`, making it easier to understand and integrate. On the other hand, Alternative 2 could become more cumbersome if additional features are required later.

The above approaches are contrasted in detail in the following table:

	Alternative 1: ...	Alternative 2: ...
Scalability	Highly scalable as additional elements can be easily integrated into the <code>state_recycler_view</code> . Simplifies future enhancements without requiring significant UI changes.	Less scalable since adding new elements requires modifying the XML file and managing multiple views separately.
Complexity	Initial implementation may be more complex due to introducing new views, viewmodels, and support functions. However, since the app already has a similar implementation (<code>state_fragment</code>), developers can reuse existing knowledge, making it easier to implement.	Initially simpler to implement as it directly involves adding views in XML. However, managing multiple views manually can increase complexity over time, especially when handling dynamic content updates.

Decision 2: How to structure `EphemeralState` that determines whether to display the Flashback Screen

I have considered the following alternatives:

Alternative 1: Introduce `CompletedState` `flashback_state` as a oneof state type in `EphemeralState`.

(In this approach, we store all the information required for the Flashback screen inside a new `CompletedState` message. However, this message would also include field like the list of `answerAndResponse`, which are not necessary for the Flashback screen and would go unused.)

Alternative 2: Introduce a `bool` `flashback_state` as a oneof state type in `EphemeralState`.

(This approach adds a simple boolean flag to indicate whether the UI should render the Flashback screen. The content needed to populate the Flashback card—such as the question and solution explanation—is already present in other fields of `EphemeralState`.)

Among these, I believe that Alternative 2 is the best approach, because:

- **Clarity & Readability:** A simple boolean flag can clearly indicate when the UI should switch to a Flashback mode. It avoids the overhead of modeling a full `CompletedState`, which may be unnecessary if no additional data is needed.

- **Efficiency:** Using a bool avoids the extra serialization and memory cost of a new message type, keeping the implementation clean and optimized.

The above approaches are contrasted in detail in the following table:

	Alternative 1: ...	Alternative 2: ...
Clarity & Readability	Adds complexity that may not be needed right now.	Simple and direct — clearly signals to show Flashback UI state.
Efficiency	Uses more storage than necessary for just a signal.	Minimal storage — only a single boolean value needed.

Decision 3: How to Show RecyclerView in Flashback Screen

I have considered the following alternatives:

Alternative 1: Introduce a new RecyclerViewAssembler class specifically for Flashback.

Alternative 2: Reuse StatePlayerRecyclerViewAssembler for the Flashback screen.

Among these, I believe that Alternative 2 is the best approach because:

- **Scalability & Maintainability:** Reusing the existing assembler avoids extra boilerplate and allows shared improvements. Future Flashback-specific changes can be handled with minor extensions.
- **Code Redundancy:** Reusing the existing StatePlayerRecyclerViewAssembler allows us to avoid unnecessary duplication and leverage already tested and stable logic.

The above approaches are contrasted in detail in the following table:

	Alternative 1: ...	Alternative 2: ...
Scalability & Maintainability	Easier to isolate Flashback-specific logic, but adds overhead and maintenance of a new class.	Promotes shared improvements and reduces boilerplate. Minor extensions can handle future needs.
Code Redundancy	Introduces duplication of existing logic.	Reduces duplication by reusing existing logic.

Risks and mitigations

Potential Risk	Mitigation
Adding new features dynamically in the conversation flow may introduce edge cases.	Implement comprehensive test cases covering different conversation scenarios to ensure smooth integration and minimize unexpected issues.
Breaking the linear learning progress by introducing features for a parallel learning experience may create inconsistencies.	Learners might get confused about where they are in the lesson if it's not clear when they're viewing a Flashback card. To avoid this, use clear visual cues—like a different colored header, a clear title, or a background that looks different from the regular lesson—to show that the learner is in Flashback mode. This helps them understand their progress in the lesson.

Key Objectives for each Milestone

Milestone	Key Objective
Milestone 1	<i>Implement Flashback Screen</i> Learners will be able to access the Flashback Card by clicking the "Revisit Previous Question" button when they get stuck on a question. After reviewing the Flashback Card, they can return to the question that caused difficulty, without having to reattempt any intermediate questions. This ensures a smoother and frustration-free learning experience. To achieve this, the following deliverables will be completed by midterm: • Feature flag for enabling Flashback functionality. • Domain logic to determine when the "Revisit Previous Question" button should be shown. • A "Revisit Previous Question" button. • A Flashback dialog that appears when the button is clicked. • The Flashback Card UI with an orange-colored toolbar.
Milestone 2	<i>Integrate Flashback Screen and Learner Conversation Flow & Improve Submitted Answers UI</i> Learners will be able to access the Flashback Card by clicking the "Revisit Previous Question" button when they get stuck on a question. When the learner clicks this button, a confirmation dialog will appear. Upon confirming, the learner is shown the Flashback Card. After reviewing the Flashback Card, learners can return to the original question without reattempting any intermediate questions. Additionally, the "Revisit Previous Question" button and feedback will be displayed in the previous responses section. Clicking "Revisit Previous Question" will allow them to revisit the Flashback Card whenever needed. Learners will also experience a redesigned submitted answer view, where multiple-choice options clearly highlight their selected response, ensuring a cleaner

	and more intuitive interface. The following deliverables will be completed by the end of the project: • The “Revisit Previous Question” button is to be directly attached to the previous answers section. • Support for the updated design of submitted answers for multiple-choice and item-selection interactions • Implementation of relevant event logs to capture user interactions with Flashback.
--	---

Implementation Approach

Domain Objects

AnswerAndResponse

Reason for change: A new String **state_name_to_revisit** will be added to support navigation to the flashback card. This `state_name_to_revisit` enables the system to determine whether to show a redirection button, such as "Revisit Previous Question". This field specifies the state to which the learner should be redirected.

```
message AnswerAndResponse {
  ...
  // Indicates that a redirection button should be shown.
  // Contains the name of the state the learner can revisit.
  string state_name_to_revisit = 4;
}
```

EphemeralState

Reason for change: The `flashback_state` boolean will be added to indicate whether the `EphemeralState` should be shown in a flashback card. When this field is set to `true`, it signals that the state is being used to present a previously shown question and the correct answer—typically linked to a learner’s misconception. This will allow the system to render the flashback card with the appropriate UI.

```
message EphemeralState {
  ...
  // Different types this state can take depending on whether the learner needs to
  // finish an existing card, has navigated to a previous card, has navigated to
  // flashback card or has reached the end of the exploration.
}
```

```

oneof state_type {
    . . .
    // Indicates whether this state is a flashback state.
    bool flashback_state = 12;

    . . .
}
. . .
}

```

ExplorationCheckpoint

Change made: No change.

Reason: The current structure of ExplorationCheckpoint already stores a MutableList of AnswerAndResponses. AnswerAndResponse now includes the newly added state_name_to_revisit. This is sufficient for restoring the learner's progress and correctly displaying the redirection button, like the "**Revisit Previous Question**" button, when a checkpoint is resumed.

PendingState

Change made: No change.

Reason: PendingState already stores a list of AnswerAndResponse objects representing the learner's previous incorrect answers and feedback. With the inclusion of the new state_name_to_revisit field in AnswerAndResponse, we will easily use the existing structure to determine whether the redirection button should be displayed.

CompletedState

Change made: No change.

Reason: Since the redirection button "Revisit Previous Question" will only be shown in PendingState and not in completed states, there's no need to modify this proto.

Solution, Interaction, HelpIndex, ExplorationCheckpoint

Change made: No changes.

Reason: These protos are unaffected by the introduction of flashback functionality and continue to serve their original purposes without requiring modification.

User Flows (Controllers and Services)

ExplorationProgressController

1. This portion of the code checks two conditions:
 - Whether the learner has submitted an incorrect answer.
 - Whether the AnswerOutcome provides a destination state that the learner has visited previously.

If both conditions are true, the destination state name will be stored in the AnswerAndResponse structure, which indicates the state player should display the “Revisit Previous Question” button.

```
private suspend fun ControllerState.submitAnswerImpl(
    submitAnswerResultFlow: MutableStateFlow<AsyncResult<AnswerOutcome>>,
    userAnswer: UserAnswer
) {
    . . .

    when {
        answerOutcome.destinationCase == AnswerOutcome.DestinationCase.STATE_NAME -> {

            // Checks whether Learner submits wrong answer and destination state name was
            // visited by Learner previously.
            if (conditions) {
                // Build an AnswerAndResponse instance with the destination state name.
                // Call setFlashbackState() in StateDeck which add this
                // AnswerAndResponse in currentDialogInteractions.
            }
        }
    }

    . . .
}
```

2. Introduce the functions moveToFlashback(), moveToFlashbackImpl(), and the data class MoveToFlashback, which notifies the controller to navigate to the flashback state. The moveToFlashbackImpl() function calculates FlashbackState and updates getCurrentState() accordingly, which enables the learner to revisit the flashback card by clicking the “Revisit Previous Question” button.

```
/**
 * [ControllerMessage] to move to the Flashback state in the exploration.
 */
data class MoveToFlashback(
```

```

    override val sessionId: String,
    override val callbackFlow: MutableStateFlow<AsyncResult<Any?>>
) : ControllerMessage<Any?>()

/**
 * Navigates to the flashback state in the graph.
 * @return a [DataProvider] indicating whether the movement to the flashback state
 * was successful.
 */
fun moveToFlashback(): DataProvider<Any?>

private suspend fun ControllerState.moveToFlashbackImpl(
    moveToFlashbackStateResultFlow: MutableStateFlow<AsyncResult<Any?>>
) {
    // get flashback state name from statedeck's AnswerAndResponse.
    // calculate ephemeralState with flashback state.
    // Updates getCurrentState().
}

```

3. Introduce the functions `moveBackToLatest()`, `moveBackToLatestImpl()`, and the data class `MoveBackToLatest`, which notifies the controller to navigate back to the latest state in the exploration. The `moveBackToLatestImpl()` function updates `AnswerAndResponse` and `getCurrentState()`, allowing the learner to return to the most recent state in the exploration after viewing the flashback card.

```

/**
 * [ControllerMessage] to move to the latest state in the exploration.
 */
data class MoveBackToLatest(
    override val sessionId: String,
    override val callbackFlow: MutableStateFlow<AsyncResult<Any?>>
) : ControllerMessage<Any?>()

/**
 * Navigates to the latest state in the graph.
 * @return a [DataProvider] indicating whether the movement to the latest state was
 * successful.
 */
fun moveBackToLatest(): DataProvider<Any?>

private suspend fun ControllerState.moveBackToLatestImpl(
    moveBackToLatestStateResultFlow: MutableStateFlow<AsyncResult<Any?>>
) {
    // update AnswerAndResponse to show "Revisit Previous Question" in Previous
    // Response section.
    // Resets getCurrentState()
}

```

```
}
```

StateDeck

1. **getFlashbackStateName()**: Returns the state to display on the flashback card. This method returns the state name from `currentDialogInteractions` that was previously stored.

```
/**
 * Returns flashback state name from [currentDialogInteractions].
 */
fun getFlashbackStateName(): String
```

2. **setFlashbackState(stateName)**: Add an `AnswerAndResponse` in `currentDialogInteractions` by setting the given `stateName`.

```
/**
 * Add AnswerAndResponse in currentDialogInteractions with the given flashback state
 * name.
 */
fun setFlashbackState(stateName: String) {}
```

Modification of PlatformParameterModule(s) to Provide the Feature Flag

With the recent migration to the new feature flag system ([PR #5725](#)), feature flags are now centrally defined in the `feature_flags.textproto` file and injected directly using the `@FeatureFlag` annotation. The `PlatformParameterModule` now only provides the `PlatformParameterController`, and does not expose individual flags.

The new **ENABLE_FLASHBACK** flag will be added to ***feature_flags.textproto*** as follows:

```
feature_flag_definition {
  id: ENABLE_FLASHBACK
  remote_server_name: "android_enable_flashback"
  default_is_enabled: false
}
```

Flags are now injected directly into classes using constructor injection via the `@FeatureFlag` annotation. For example:

```
@FeatureFlag(ENABLE_FLASHBACK)
private val enableFlashback: Boolean
```


This allows classes to access the current flag value at runtime, simplifying the logic for conditional feature behavior.

The TestPlatformParameterModule will be extended to support setting the ENABLE_FLASHBACK flag explicitly during tests. This ensures that both enabled and disabled paths for Flashback functionality are testable.

The Flashback feature will be gated behind the ENABLE_FLASHBACK flag in multiple components, including UI, logic, and controllers:

Gated Components

1. XML-level gating via data binding:
 - question_options_recycler_view visibility toggled based on ENABLE_FLASHBACK.
2. ViewModel-level logic:
 - SelectionInteractionViewModel uses the flag to decide whether to run convertSelectedItemsToHtmlString() for Flashback behavior.
3. Controller-level logic:
 - If the flag is enabled, a redirection button will be activated during exploration sessions to support the Flashback behavior.

Tests must verify behavior for both feature flag ON and OFF states.

UI changes

New UI Components & XML Layouts

We will need to introduce the following UI components along with their XML layouts and corresponding ViewModels:

Redirection buttons

- **Revisit Previous Question button** - *introduce flashback_button_item.xml, FlashbackButtonViewModel, addRedirectionSupport().*
- **Return to question button** - *introduce return_to_question_button_item.xml, ReturnToQuestionViewModel, addRedirectionSupport().*

Flashback card UI Components

- **Feedback** - *use existing feedback_item.xml, FeedbackViewModel, addFeedbackSupport().*
- **Previous questions** - *use existing content_item.xml, ContentViewModel, addContentSupport(),*
- **Solution section** - *use solution_summary.xml with some changes and introduce support function addSolutionSupport() in statePlayerRecyclerViewAssembler.*
- **Submitted Answer** - *use existing submitted_answer_item.xml, SubmittedAnswerViewModel, and addPastAnswerSupport(), but need some changes to support enhanced look and ui.*

- **Return to question button** - introduce `return_to_question_button_item.xml`, `ReturnToQuestionViewModel`, `addRedirectionSupport()`.
- **New Toolbar**- introduce a new toolbar for the flashback card with a different color and title in `exploration_activity.xml`.

We will need to modify `StatePlayerRecyclerViewAssembler` to support newly introduced views and buttons. We will update the `compute()` method to add the new `ViewModels` into the `itemList` for proper rendering.

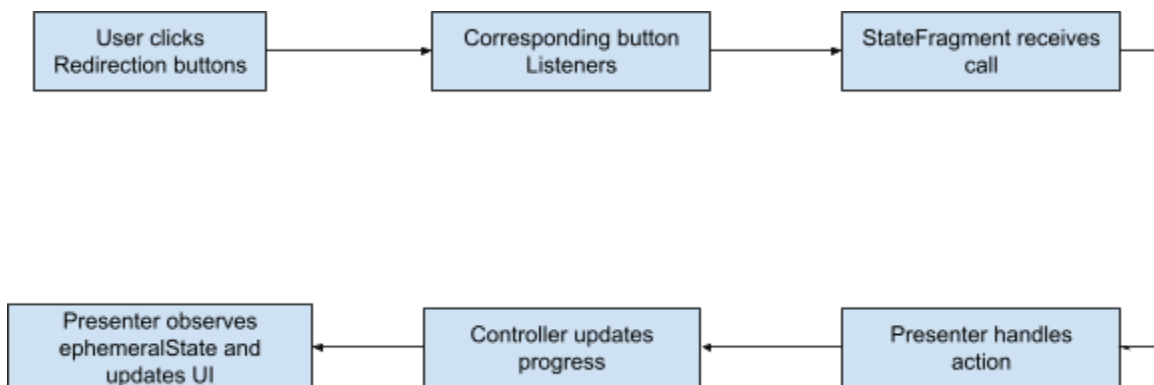
Also, necessary updates will be implemented in `addPreviousAnswers()` to add the "Revisit Previous Question" button and ensure proper feedback handling and other required methods. Also, new support methods will be introduced for rendering UI components in the Flashback screen.

We will introduce Listener Interfaces for Redirection buttons. The `StateFragment` will be responsible for implementing these listeners.

```
/** Listener for when the 'Revisit Previous Question' button is clicked. */
interface RevisitPreviousQuestionButtonListener {
    fun onRevisitPreviousQuestionButtonClicked()
}

/** Listener for when the 'Return to Question' button is clicked. */
interface FlashbackNavigationListener {
    fun onReturnToQuestionButtonClicked()
}
```

Listeners need to be added to handle user interactions when users click on redirection buttons, such as viewing a Flashback card or returning to the latest card.



Flashback Screen

To support flashback mode, we will update the `state_fragment.xml` to display the flashback card using the `state_recycler_view`. Additionally, we will introduce a new toolbar in

[exploration_activity.xml](#) with a distinct title and color to visually differentiate flashback mode from the regular exploration experience.

We will also define the necessary views, ViewModels, and helper functions required to render the flashback card UI. The [compute\(\)](#) method of `StatePlayerRecyclerViewAssembler` will be utilized to generate the appropriate list of ViewModels based on the `ephemeralState` corresponding to the flashback card.

Enhancing the Look and Feel of the Submitted Answer in FlashbackCard

To enhance the submitted answer look, we will need to introduce XML files and their viewmodels and modify existing XML files.

- We will add a RecyclerView (`question_options_recycler_view`) inside [submitted_answer_item.xml](#). This will allow a structured and dynamic display of submitted answers, improving readability.
- We will create layouts for multiple-choice submitted answers and item-selection submitted answers. This includes new XML layouts and viewmodel.
 - `multiple_choice_submitted_item.xml`, `item_selection_submitted_item.xml`, and `SelectionSubmittedItemViewModel`. These layouts will be similar to [multiple_choice_interaction_items.xml](#) and [item_selection_interaction_items.xml](#).

```
/**
 * ViewModel representing a submitted answer in a selection interaction.
 *
 * This ViewModel holds information about a selectable choice and whether it is
 * correct
 * @property htmlContent The HTML content representing the choice text.
 * @property hasConversationView Boolean flag indicating if it's in conversation
mode.
 * @property itemIndex The index of the choice in the list.
 * @property isCorrect Boolean indicating if this choice is a correct answer.
 * @property customTagHandlers Map of custom HTML tag handlers for rendering.
 * @property writtenTranslationContext The translation context for displaying
translated text.
 * @property translationController Controller managing translations.
 */
class SelectionSubmittedItemViewModel(
    val htmlContent: SubtitledHtml,
    val hasConversationView: Boolean,
    val itemIndex: Int,
    val isCorrect: Boolean,
    val customTagHandlers: Map<String, CustomHtmlContentHandler.CustomTagHandler>,
    val writtenTranslationContext: WrittenTranslationContext,
    val translationController: TranslationController
)
```

)

- Data handling in SubmittedAnswerViewModel will be enhanced by computing List<SelectionSubmittedItemViewModel> based on interaction.id for the flashback card. Correct options will be marked to improve user understanding. The computed list of answer items will then be set in SubmittedAnswerViewModel. A similar functionality is also present in the [SelectionInteractionViewModel](#).

Finally, question_options_recycler_view will retrieve the list from SubmittedAnswerViewModel and display the submitted answers with proper formatting.

```
/**
 * Computes a list of choice items marked with correct answers.
 *
 * This function generates an `ObservableArrayList` of
 * `SelectionSubmittedItemViewModel` instances, each representing a choice,
 * highlighting the correct ones.
 */
private fun computeSubmittedItems(
    choiceSubtitledHtmls: List<SubtitledHtml>,
    hasConversationView: Boolean,
    selectionInteractionViewModel: SelectionInteractionViewModel,
    correctAnswerIndexes: List<Int>,
    writtenTranslationContext: WrittenTranslationContext,
    translationController: TranslationController,
    customTagHandlers: Map<String, CustomHtmlContentHandler.CustomTagHandler>
): List<SelectionSubmittedItemViewModel>
```

Additionally, we will remove the precomputation of HTML answer strings from the [SelectionInteractionViewModel](#), which is currently responsible for rendering submitted answers for both multiple-choice and item-selection interactions. Specifically, the method [convertSelectedItemsToHtmlString](#) will be removed. Instead, question_options_recycler_view will handle the structured display of submitted answers for both multiple-choice and item-selection interactions.

(Note: The question_options_recycler_view will be used in addition to the existing submitted_answer_recycler_view. It will be gated behind the ENABLE_FLASHBACK flag. When the flag is ON, question_options_recycler_view will be shown).

Test data changes

A new or modified test exploration will be required to effectively demonstrate the Flashback feature, ensuring it covers scenarios where learners get stuck on a question and they revisit a Flashback Card. The learner should be able to return to the original question without reattempting intermediate steps.

Testing library changes

The following files will require an update of existing test cases or the addition of new tests:

- ExplorationActivityTest
- ExplorationProgressControllerTest
- StateFragmentTest

Script & CI changes

The project does not require any Script & CI changes.

Documentation changes

The project does not require any Documentation changes.

Metrics Plan

Event (see PRD)	Event parameters (see PRD)	Do we already record the event + parameters? • If so, please link to the corresponding code on GitHub. • If not, describe the changes needed to do so.
open_flashback_card_event	exploration_id, state_name, state_name_to_revisit	We need to create a new FlashbackCardContext in the event log message, which will cascade down to the ActivityContextCase in the EventBundleCreator. To implement this, we should log the event from the ExplorationProgressController when the normal ephemeral state transitions to a flashback state. New convenience method will be added to the LearnerAnalyticsLogger to be called from the explorationProgressController to log an event with a FlashbackContext.
close_flashback_card_event	exploration_id, state_name, state_name_to_revisit	We should log the event when the learner closes the flashback and returns to the main question, which occurs when the "Return to question" button. To implement this, we should log the event from the ExplorationProgressController when the flashback state transitions to a normal ephemeral state. New convenience method will be added to the LearnerAnalyticsLogger to be called from the explorationProgressController to log an event with a FlashbackContext.
flashback__button_event	classroom_id, topic_id, story_id, exploration_id,	Will be of type ExplorationContext . This event log will be used to track the click event of "Revisit Previous

	session_id, exploration_version, state_name, learner_details	Question” button.
--	---	-------------------

Implementation Plan

Milestone Table (include both PRs and other actions that need to be taken prior to launch)

Milestone 1 (Jun 2 to Jul 4)

Key objective for this milestone: *Implement Flashback Card and hook up with the “Revisit Previous Question” button.*

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
1.1	Introduce a feature flag for the Flashback feature.	None	Jun 3	Jun 5
1.2	Implement logic to determine when to show “Revisit Previous Question” button for viewing the flashback card, and add relevant tests.	None	Jun 6	Jun 9
1.3	Introduce new UI component for “Revisit Previous Question” button and add tests.	1.2	Jun 10	Jun 13
1.4	Determine the appropriate state to show in the Flashback screen, implement navigation logic to view the Flashback Card, and add tests.	1.3	Jun 16	Jun 21
1.5	Introduce the UI components for Flashback Screen, integrate with the “Revisit Previous Question” button, and add relevant tests.	1.4	Jun 24	Jun 27
1.6	Implement navigation logic for returning to the pending question after viewing Flashback and include tests.	1.5	Jun 27	Jun 30
1.7	<u>Demo to PMs/stakeholders:</u> <i>Flashback Screen, navigation flow, and seamless redirection without duplicate entries.</i>	1.6	Jul 8	

Milestone 2 (Jul 26 to Aug 27)

Key objective for this milestone: *Integrate Flashback Card and Learner Conversation Flow & Improve Submitted Answers UI.*

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
2.1	Implement logic to display the “Revisit Previous Question” button. Enhance conversation flow - Add “Revisit Previous Question” button , - Feedback in previous responses section. - Add tests.	1.5	Aug 1	Aug 6
2.2	Update the design of submitted answers for multiple-choice and item-selection interactions and add tests.	None	Aug 13	Aug 18
2.3	Capture and log events from the new Flashback screen.	2.1	Aug 16	Aug 19
2.4	Add additional tests in StateFragmentTest.	2.3	Aug 20	Aug 24
2.5	<u>Demo to PMs/stakeholders:</u> <i>Showcase the enhanced Flashback experience, improved submitted-answer UI.</i>	2.4	Sep 2	

Feature Flags

#	Name of feature flag	What happens when the flag is turned on	Needs sync between Android / Web?	Key stakeholders
1.	ENABLE_FLASHBACK	Enables the Flashback feature, allowing learners to revisit key concepts without reattempting all intermediate questions.	No.	Android CLaM, Product

Future Work

Future work could focus on expanding the Flashback feature to support more complex learning paths and enhancing analytics for deeper insights into learner engagement. Additional improvements may include refined UI/UX optimizations and enhanced accessibility for a more inclusive experience.